

ICS 214 Lab 3

I wanted to preface this lab with an introduction or recap on using the Visual Studio debugger (depending on if we've covered it in class). I have noticed in prior years that many students are not familiar with using a debugger. Knowing how to use it can be so helpful – especially when dealing with runtime errors. The first section of this lab serves as an intro (or reminder) on how to use some of the key aspects of the debugger. If you're already familiar with setting breakpoints, stepping through code (into and out of functions), and viewing the contents of variables, you can skim it and skip to the section below titled "Coupon Collector's Problem".

If you are not using Visual Studio you will likely find that there is a similar type of debugger that comes with the IDE you are using, and you'll probably want to google the hotkeys that allow you to step, step in, step out of functions and view the contents of variables.

Debugging with Visual Studio

Knowing how to debug your code is a key part of development. The debugger gives you a window into exactly what is going on with your code and helps you to understand exactly how your program works.

Debugging usually involves

1. Setting breakpoints
2. Stepping through code
3. Observing the contents of variables in memory.

In order to debug our code, we need to first be able to diagnose what is wrong with our program. That part can sometimes be a little tricky (particularly when reproducing a known bug is difficult).

Breakpoints

What are breakpoints? A breakpoint marks a point in our program at which the debugger will pause. We can set a breakpoint on any line of code in our program and pause the execution in order to examine the contents of variables, and of memory in general. It will suspend the entire program and allow us to observe the state of a program. Being able to view memory, is incredibly useful in diagnosing issues with your program (if they exist).

By viewing memory, you can view the value of every variable and determine "Hey, this variable doesn't contain what it should... something is obviously wrong." You can also step through the execution of your program line by line. Eg: You can put a breakpoint on line 9, run your program until that exact line is about to execute, and then hit a key (F10) which will advance

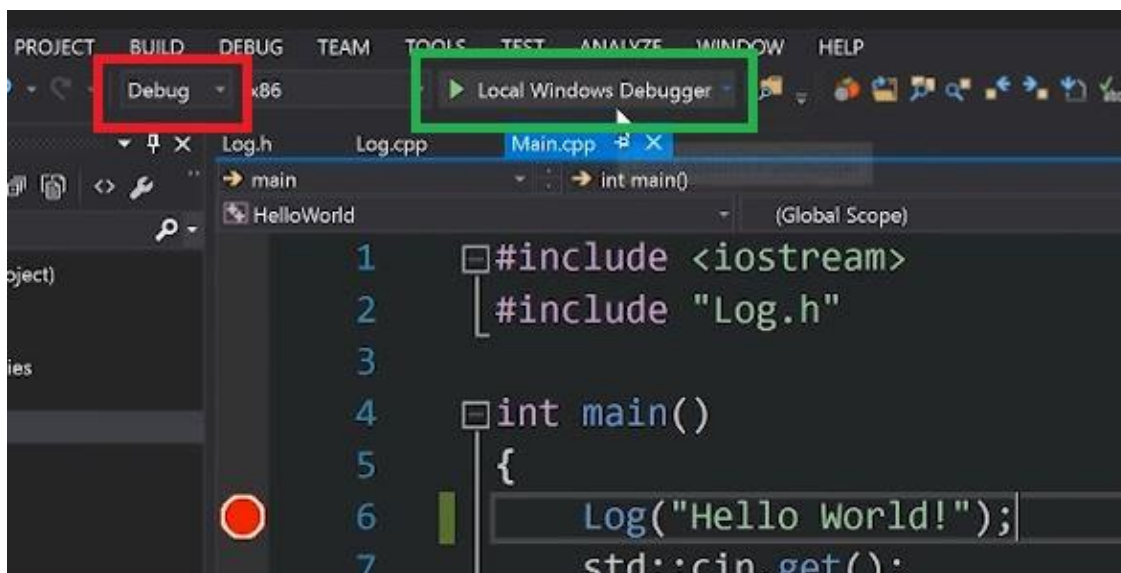
the program one line further. You can also step in (F11) and out (SHIFT+F11) of functions (once we start using them) to determine where they take you, and what they do.

To set a breakpoint in Visual Studio, you can either hit F9 to pause (“break”) on the line your cursor is on, or you can left click on the grey vertical bar on the left side of the page. You should see a red dot appear.



Setting a breakpoint on an empty line is not going to be very helpful, so make sure you set a breakpoint on a line where code will be executed.

You should be in Debug mode (by default). Ensure you can see the red box in image below), because if you are in Release mode, the compiler will actually change the way that you code looks, and your breakpoints may never get hit because your program has been re-arranged.



Once you are ready to go, hit the “Local Windows Debugger” button (green box in image), or hit F5 to run your program.

Once your program reaches the point where the breakpoint is, it will pause, and Visual Studio will change to have a bit of an alternate layout (with some new windows: Watch, Call Stack, etc...). You will be able to see where the program is paused because it puts a big yellow arrow in the left column to indicate where the program’s instruction pointer is currently at. The line of

code that the arrow is currently pointing to has not executed yet (so any instructions that are on that line have not yet happened – until you step past that line).

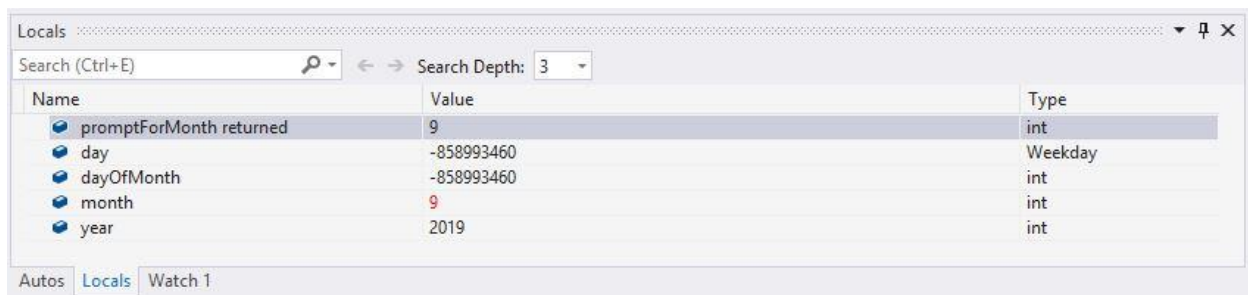


Now that our program is paused, we can take the opportunity to look at the values that different variables contain.

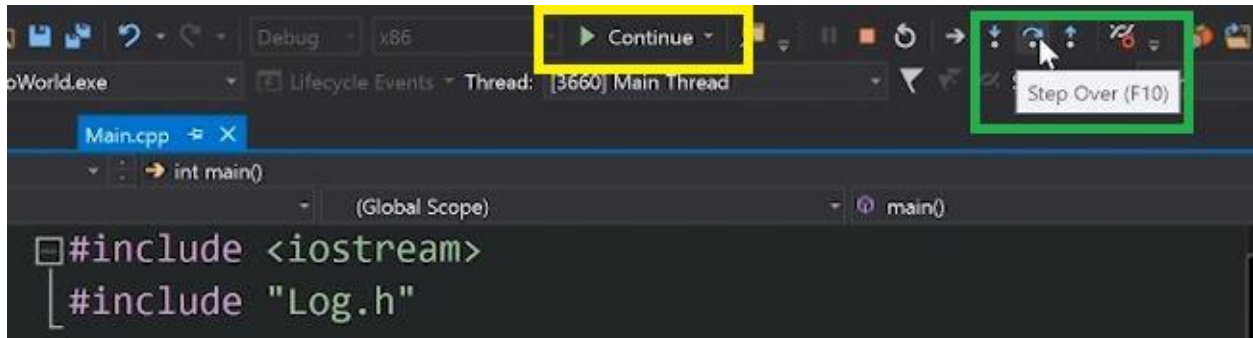
Looking at Memory

Once your program has stopped on a breakpoint, you will be able to see the contents of the variables in your code simply by hovering the mouse cursor over the variable names. Try this when you are working through this lab.

A second way to observe the value of a variable, is to observe the locals window (typically located in the lower left corner). This window allows you to keep a constant eye on all the local variables and their contents as you step through code (without having to roll over the variable name). If the locals window is not available by default, place a breakpoint on the first line of main, start debugging (F5), and then select Debug | Windows | Locals – to make it visible. Observe changes to the contents of variables (and even the return values of functions) as you advance through the code (using F10 – Step Over – from above).

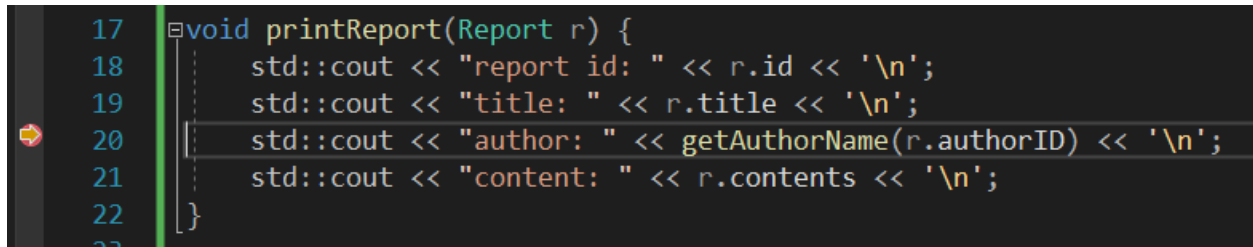


Once you have had a chance to look at the contents your variables and you want to continue, the following buttons will allow us to choose what happens next with our program.



Clicking **Continue** (yellow box), or hitting **F5**, will allow us to resume running the program.

The other 3 buttons (arrow buttons inside the green box) will allow us to step through our program one line at a time.



“Step Over” (F10) will execute `getAuthorName()` and step to the next line in the program (line 21) where execution will be paused.

“Step Into” (F11) will allow us to step into a function (in this case, the `getAuthorName()` function). Execution will be paused on the first line of `getAuthorName()`. We could then use F10 to step through each line in `getAuthorName()`.

“Step Out” (SHIFT-F11) would execute the code from the current line to the end of the function and return to the “client” that called it. Execution be paused on the line of code directly after `printReport()` was called (by the client).

In setting breakpoints and stepping through our program, we can literally see the execution of our program – line by line. This is so useful, especially when you are getting results that you are not expecting, and you are trying to figure out what you have done wrong!

Debugging Summary:

This is the basis of how we debug our code. Remember, a program is just made up of code and data stored in memory. Being able to view what is in memory is a huge asset.

Being able to set breakpoints allows us to pause execution on any given line of code and inspect the values our variables contain. That alone, can be incredibly useful when you are running code, especially if you are trying to figure out why something is not as you expect.

Attempt to use the debugger if you encounter something in your code that isn't working as expected. The more familiar you are with this tool, the easier (and faster) it will be to fix problems.

Objectives:

1. Fixed arrays – creating, using, and passing to functions.
2. Use enum classes
3. Breaking logic into functions
4. Using static casting
5. Use prototypes (forward declarations) to tell the compiler what our function declarations look like.

Coupon Collector's Problem

Create a new empty project called Lab3. Add a source file and call it Main.cpp.

Overview:

Simulation: coupon collector's problem:

Coupon collector is a classic statistic problem with many practical applications. The problem is to pick objects from a set of objects repeatedly and determine how many picks are needed for all the objects to be picked at least once (on average).

Default Arguments

One of the functions you are asked to write contains a default argument. This concept should have been covered in Java. If it seems unfamiliar, read through [LearnCpp section 11.5 \(Default arguments\)](#).

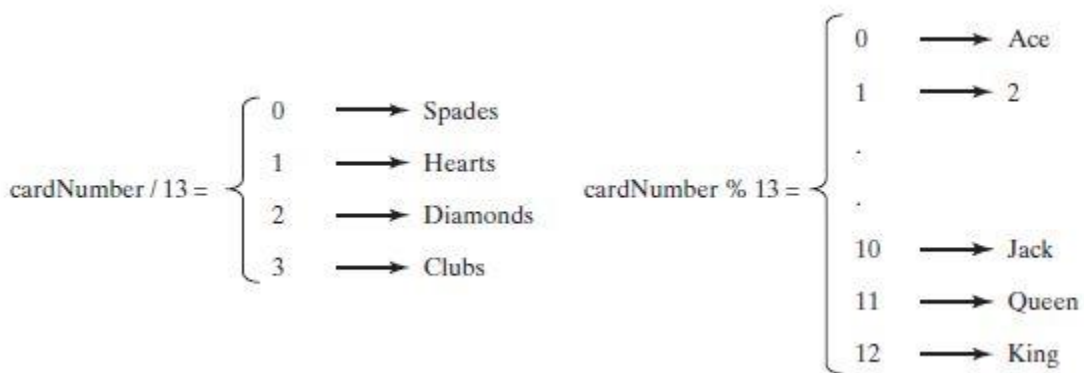
Note that a default argument can't be specified in both a .h file, and a .c file, because it presents opportunity for 2 conflicting defaults. It can only be specified in one place. You'll want to specify it in the .h file. That is usually where a programmer will look to find out how a function works.

Task 1:

A variation of the coupon collector's problem is to pick cards from a shuffled deck of 52 cards repeatedly and find out how many picks are needed before you see one of each suit (hearts,

clubs, diamonds, spades). Assume that a picked card is placed back in the deck before picking another. Write a program to simulate the number of picks needed to get one card from each suit (and display the four cards picked). Since picked cards are placed back in the deck, it is possible that a card may be picked twice.

We can simulate picking a card by generating a card number between [0-51] and then determining its rank and suit as shown below:



Below is the desired output from a sample run of Task 1. Note: once a suit has been picked, the program should not output subsequent picks from that same suit.

```
queen of spades
five of clubs
queen of hearts
four of diamonds
Number of picks: 12
```

Create Main.cpp

For Task 1, our main() function is only going to do 2 things:

1. Seed the randomizer
2. Output the number of picks returned when we call `getPickCountNeededForFourSuits()` (all the logic for determining how many cards we need to draw before we have seen 4 suits resides in this function).

Create a Cards.h file. Copy the code below into it and then add header guards for Cards.h

```
enum class Suit {};           // this will need to be filled out
enum class Rank {};          // this will need to be filled out

namespace Constants {
    constexpr int CARD_COUNT{ 52 }; // the # of cards in a deck
    constexpr int NUM_RANKS{ 13 };  // the # of ranks in a deck (two - ace)
    constexpr int NUM_SUITS{ 4 };    // the # of suits in a deck (hearts, clubs,
etc.)
```

```

    // These string arrays will simplify the task of printing out the card values.
    // We don't need to use a switch statement to find the string representation
    // of an enum. We can cast the enum as an int to index these arrays.
    //const std::string SUITS[] {}; // initialize & match with enums
    //const std::string RANKS[] {}; // initialize & match with enums
}

```

[Optional tip]: A helpful trick that some programmers do with enums is to add a last entry to represent the number of items in the enumeration. For example: if I add the word “count” as the last entry in the Suits enumeration, I can use `static_cast<int>(Suit::count)` in code. This will be 4 since the actual suits are enumerated from 0 to 3.

This has two main benefits:

1. You can eliminate the NUM_SUITS constant (use `Suit::count` instead)
2. If you ever add/remove an entry to the enum, the count will automatically be updated (without it, I have to remember to update NUM_SUITS manually).

Create a `Cards.cpp`. Copy the code below into it.

```

// Pick a random card from the deck (represented by an int between 0-52)
// - params: none
// - return: an int between 0 - 51
// pickRandomCard();

// Get the rank of a specific card index
// - param 1: an int representing the card index
// - return: an enum representing the Rank of the card index given.
// getRank();

// Get the suit of a specific card index
// - param 1: an int representing the card index
// - return: an enum representing the Suit of the card index given.
// getSuit();

// A function to assess whether all elements in a boolean array are true
// You can use this function to determine whether all suits have been picked.
// - param 1: an array of boolean values(decide if it should be const or not)
// - param 2: ? do we need any other parameters here to make this work? You decide.
// - return: a bool : true if ALL the elements in param 1 are true, false otherwise.
// allArrayElementsAreTrue();

// This is the function that does all the work behind solving the problem (including
// sending output to the console).
// This function should create/use an array of Boolean values (all initially false)
// to represent the suits that have been picked.
// It should make use of allArrayElementsAreTrue() to test if all suits have been
// picked.
// - param 1: a bool called “verbose” (meaning wordy) that defaults to true.
//             If verbose is true, generate output cards picked & the pick count.
// - return: an int representing the number of card picks it takes to cover 4 suits.
// getPickCountNeededForFourSuits();

```

#include "Cards.h" at the top of Cards.cpp and Main.cpp file.

Step through the implementation of each function (in order) in Cards.cpp. Once you decide on the function signature, add the forward declaration (and comments) to Cards.h. This will allow you to test each function from main() as you write it.

Task 2:

This second task is based on a "Monte Carlo simulation" and builds on task 1. It should only take 5 or 10 minutes. The idea behind this is that a random sample represents properties exhibited by greater population.

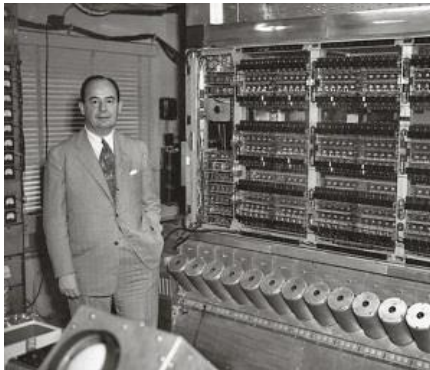
The Monte Carlo simulation uses random numbers and probability to solve problems. This method has a wide range of applications in computational mathematics, physics, chemistry, and finance.

It was invented by Stanislaw Ulam (a Polish-American Mathematician, known for developing thermonuclear weapons) who helped to build the atomic bomb.

Stanislaw got help from John von Neumann (inventor of stored program computer) to solve a probability problem in relation to playing solitaire. He went on to use the Monte Carlo simulation to help with the design of the hydrogen bomb (to investigate the distance neutrons would travel through various materials).



Stanislaw Ulam



John von Neumann

If you are keen on learning more about the Monte Carlo Simulation, there is an interesting MIT university lecture on it at: <https://www.youtube.com/watch?v=OgO1gpXSUzU&t=1516s>

We want to use this concept to estimate an **average** of how many cards you need to draw to see all 4 suits. The more iterations you do, the more accurate your result will be.

Change your main() and create a simple for loop. Call the getPickCountNeededForFourSuits() function created above. You will want to set the verbose parameter to false – otherwise you will get a huge amount of output which will take forever to output to the console. Create

variables for keeping track of the total cards drawn and the total “trials” (iterations) run. Calculate the average by dividing totalCardsDrawn by totalTrials and output the results.

Try it with a loop size of 100 to begin with... and work your way to 100,000,000. What value did you find the simulation approached as your iterations increased? (if it's not 8.333... there's probably something wrong with your calculation)

Task 3

Often in game programming, there are situations where you have a randomized set of items, and you need to draw items out of the set (one by one) until it is empty. At that point the set is usually repopulated with the original items, re-randomized, and the process begins again. This is called random sampling without replacement.

A good example of this would be a bingo machine that contains balls (1-75). Balls are drawn (without replacement) until the game is won, the ball holder is repopulated, and the process begins again.

In this lab (so far), we wrote a function: pickRandomCard(), which simulated random sampling WITH replacement (cards are replaced in the deck). Random sampling without replacement is more difficult because you need to ensure the things you have chosen already aren't picked again.

Write new functionality (or extend existing functionality to implement the exact same averaging test where cards are drawn randomly without replacement). Calculate how many picks would be required (on average) when we don't put the picked cards back into the deck (until all 4 cards have been seen). If you plan to modify a function that is currently working – it is a good idea to duplicate the function and comment it out so you have a backup and can always revert to the working version.

There are multiple ways of going about this. If you're unsure of where to start – I've put some hints on the next page.

The average for my random without replacement calculation narrowed to an average of 7.66497 picks – slightly less than the 8.3333 average that resulted from a random pick with replacement. This was based on calling the function 100 million times in a loop (note: this took over 15 minutes on an older computer, but I've seen it will work much faster in the lab) - 10 million iterations, should be close enough to get within 2 decimal points (7.66). If you're interested in the formula for determining the solution using math, you can find it here:

<https://math.stackexchange.com/questions/268432/expected-number-of-card-draws-to-get-all-4-suits>

It was also interesting to note the speed difference between 10 million iterations using random with replacement (3 seconds), vs random without replacement (5 to 20+ seconds depending on how you do it).

Hints for Task 3:

If you're stuck on how to go about this here are some hints about how I implemented a solution. I originally started writing an entirely new function called `getPickCountNeededForFourSuitsRNR()` (random no replacement). I noticed however that most (80%) of the code in it was the same as the code in `getPickCountNeededForFourSuits()`. I then took a closer look at the function to see what part needed to change to make things work. The only thing that really needed to change is how the card index is randomly drawn – currently encapsulated in `pickRandomCard()`. So, I decided to go a different route outlined in the tips below. Feel free to use them or to go a totally different direction.

- You could modify the existing `getPickCountNeededForFourSuits()` by adding a bool parameter: “useReplacement” that defaults to true and could be checked to determine how to choose a random card. I found a lot of duplication was avoided by modifying the existing function.
- If “useReplacement” is true: pick cards indices using `pickRandomCard()`
- If “useReplacement” is false: you still want to pick a random card, but you want to do so in a way that prevents you from picking a card you have picked before. Here are a couple ideas for how to do that (one method is significantly faster than the other, but either should work):
 1. Use an array of ints to represent the card indices you could pick [0...51]. Shuffle it using `random_shuffle` (<https://stackoverflow.com/questions/14720134/is-it-possible-to-random-shuffle-an-array-of-int-elements>), and then use an int to step through the array starting at index 0. This will guarantee you won't see the same card more than once. Using `random_shuffle` may seem a little bit foreign (because we haven't discussed iterators at all), but you should be able to make things work by looking at the examples.
 2. Use an algorithm like the one you used to track which suits have been seen. Instead of using a bool array of 4 suits, use a bool array of 52 (for each card index). Randomly pick a card index, and if you've seen it already – keep picking!

Optimization:

Lastly, optimize the `getPickCountNeededForFourSuits()` function to make it run as quickly as possible. It is being called 10 million times! If it does the minimum it needs to do, then it will run much faster. If it does more, then it will take longer. When you do a code review, compare your solution's completion time with your reviewer's completion time.

For example:

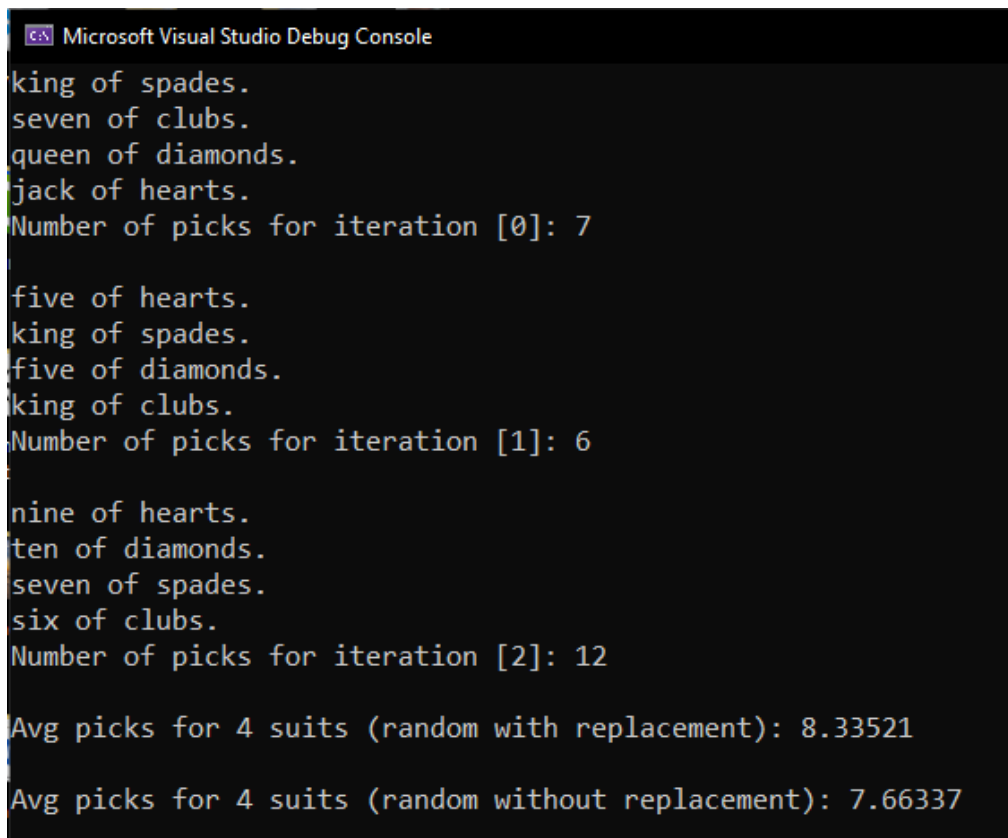
- If you are using replacement, you won't need to initialize or shuffle a deck (since you are picking random integers to determine card numbers).
- If you aren't printing to the console (`verbose` is false), you won't need to calculate rank, you also won't need to do anything with strings (string processing can take a lot of time)!

Demo preparation:

Set up your code to demo the results of both Task 1 and Task 2 in the same run (so it takes less time to demo)

- For Task 1: make 3 calls to `getPickCountNeededForFourSuits()` (with verbose mode on).
- For Task 2, execute the loop (10,000,000 times with verbose mode off), and output the results (average picks from all the loop iterations).

Your output should look like the image below:



```
Microsoft Visual Studio Debug Console
king of spades.
seven of clubs.
queen of diamonds.
jack of hearts.
Number of picks for iteration [0]: 7

five of hearts.
king of spades.
five of diamonds.
king of clubs.
Number of picks for iteration [1]: 6

nine of hearts.
ten of diamonds.
seven of spades.
six of clubs.
Number of picks for iteration [2]: 12

Avg picks for 4 suits (random with replacement): 8.33521
Avg picks for 4 suits (random without replacement): 7.66337
```

Code Review

When you are finished this assignment, find a friend who is also finished. Conduct a code review on each other's code **before** giving a demo of your assignment for marks. Compare execution speed to see if you've optimized to the same degree that your partner has optimized.

Marking:

- Does it work as expected?
- Are variables in local scope instead of global?
- Is the scope of local variables as small as possible (declared when used)?
- Are there comments to explain anything non-trivial or not obvious?
- Have constants been used instead of "magic numbers" in the code.
- Have header guards been added to .h files?
- Have you optimized your code? Is there work being done that doesn't need to be done in some cases? The loop runs millions of times, so you can shorten the time it takes by ensuring calculations only happen when necessary.

Task 1 [6/10]

- Output shows only the first card of each suit
- Verbose flag (true/false) works properly

Task 2 [2/10]

- Result narrows to ~8.33 for 10 million iterations

Task 3 [2/10]

- Result narrows to ~7.66 for 10 million iterations

Labs are due a week from when they are assigned (at midnight). Late labs will incur a penalty of 20% per week.